Chapter 4 : Intermediate SQL

Database System Concepts, 7th Ed.

Outline

- Join Expressions
- Views
- Transactions
- Integrity Constraints
- SQL Data Types and Schemas
- Index Definition in SQL
- Authorization

Joined Relations

- **Join operations** take two relations and return as a result another relation.
- A join operation is a Cartesian product which requires that tuples in the two relations match (under some condition). It also specifies the attributes that are present in the result of the join
- The join operations are typically used as subquery expressions in the **from** clause
- Three types of joins:
  - Natural join
  - Inner join
  - Outer join

Natural Join in SQL

- Natural join matches tuples with the same values for all common attributes, and retains only one copy of each common column.
- List the names of instructors along with the course ID of the courses that they taught
  - **select** *name*, *course_id*
    from *students, takes*
    where *student.ID = takes.ID*;
- Same query in SQL with “natural join” construct
  - **select** *name*, *course_id*
    from *student* **natural join** *takes*;

Natural Join in SQL (Cont.)

- The **from** clause can have multiple relations combined using natural join:

  select $A_1, A_2, \ldots A_n$
  from r_1 **natural join** r_2 **natural join .. natural join** r_n
  where P ;

Student Relation

ID	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

Takes Relation

ID	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2017	A
00128	CS-347	1	Fall	2017	A-
12345	CS-101	1	Fall	2017	C
12345	CS-190	2	Spring	2017	A
12345	CS-315	1	Spring	2018	A
12345	CS-347	1	Fall	2017	A
19991	HIS-351	1	Spring	2018	B
23121	FIN-201	1	Spring	2018	C+
44553	PHY-101	1	Fall	2017	B-
45678	CS-101	1	Fall	2017	F
45678	CS-101	1	Spring	2018	B+
45678	CS-319	1	Spring	2018	B
54321	CS-101	1	Fall	2017	A-
54321	CS-190	2	Spring	2017	B+
55739	MU-199	1	Spring	2018	A-
76543	CS-101	1	Fall	2017	A
76543	CS-319	2	Spring	2018	A
76653	EE-181	1	Spring	2017	C
98765	CS-101	1	Fall	2017	C-
98765	CS-315	1	Spring	2018	B
98988	BIO-101	1	Summer	2017	A
98988	BIO-301	1	Summer	2018	*null*

student natural join *takes*

ID	*name*	*dept_name*	*tot_cred*	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	Zhang	Comp. Sci.	102	CS-101	1	Fall	2017	A
00128	Zhang	Comp. Sci.	102	CS-347	1	Fall	2017	A-
12345	Shankar	Comp. Sci.	32	CS-101	1	Fall	2017	C
12345	Shankar	Comp. Sci.	32	CS-190	2	Spring	2017	A
12345	Shankar	Comp. Sci.	32	CS-315	1	Spring	2018	A
12345	Shankar	Comp. Sci.	32	CS-347	1	Fall	2017	A
19991	Brandt	History	80	HIS-351	1	Spring	2018	B
23121	Chavez	Finance	110	FIN-201	1	Spring	2018	C+
44553	Peltier	Physics	56	PHY-101	1	Fall	2017	B-
45678	Levy	Physics	46	CS-101	1	Fall	2017	F
45678	Levy	Physics	46	CS-101	1	Spring	2018	B+
45678	Levy	Physics	46	CS-319	1	Spring	2018	B
54321	Williams	Comp. Sci.	54	CS-101	1	Fall	2017	A-
54321	Williams	Comp. Sci.	54	CS-190	2	Spring	2017	B+
55739	Sanchez	Music	38	MU-199	1	Spring	2018	A-
76543	Brown	Comp. Sci.	58	CS-101	1	Fall	2017	A
76543	Brown	Comp. Sci.	58	CS-319	2	Spring	2018	A
76653	Aoi	Elec. Eng.	60	EE-181	1	Spring	2017	C
98765	Bourikas	Elec. Eng.	98	CS-101	1	Fall	2017	C-
98765	Bourikas	Elec. Eng.	98	CS-315	1	Spring	2018	B
98988	Tanaka	Biology	120	BIO-101	1	Summer	2017	A
98988	Tanaka	Biology	120	BIO-301	1	Summer	2018	*null*

Dangerous in Natural Join

- Beware of unrelated attributes with same name which get equated incorrectly
- Example -- List the names of students instructors along with the titles of courses that they have taken
  - Correct version

    select *name*, *title*
    from *student* **natural join** *takes*, *course*
    where *takes.course_id* = *course.course_id*;

  - Incorrect version

    select *name*, *title*
    from *student* **natural join** *takes* **natural join** *course*;

    - This query omits all (student name, course title) pairs where the student takes a course in a department other than the student's own department.
    - The correct version (above), correctly outputs such pairs.

Natural Join with Using Clause

- To avoid the danger of equating attributes erroneously, we can use the "**using**" construct that allows us to specify exactly which columns should be equated.
- Query example

  select *name*, *title*
  from (*student* **natural join** *takes*) **join** *course* **using** (*course_id*)

Join Condition

- The **on** condition allows a general predicate over the relations being joined
- This predicate is written like a **where** clause predicate except for the use of the keyword **on**
- Query example

  select *
  from *student* **join** *takes* **on** *student_ID = takes_ID*

  - The **on** condition above specifies that a tuple from *student* matches a tuple from *takes* if their *ID* values are equal.
- Equivalent to:

  select *
  from *student , takes*
  where *student_ID = takes_ID*

Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples form one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values.
- Three forms of outer join:
  - left outer join
  - right outer join
  - full outer join

Outer Join Examples

- Relation *course*

course_id	*title*	*dept_name*	*credits*
BIO-301	Genetics	Biology	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3

- Relation *prereq*

course_id	*prereq_id*
BIO-301	BIO-101
CS-190	CS-101
CS-347	CS-101

- Observe that

  course information is missing CS-437

  prereq information is missing CS-315

Left Outer Join

- *course* **natural left outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*

- In relational algebra: *course* ⟕ *prereq*

Right Outer Join

- *course* **natural right outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	*null*	*null*	*null*	CS-101

- In relational algebra: *course* ⟖ *prereq*

Full Outer Join

- *course* **natural full outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*
CS-347	*null*	*null*	*null*	CS-101

- In relational algebra: *course* ⟗ *prereq*

Joined Types and Conditions

- **Join operations** take two relations and return as a result another relation.
- These additional operations are typically used as subquery expressions in the **from** clause
- **Join condition** – defines which tuples in the two relations match.
- **Join type** – defines how tuples in each relation that do not match any tuple in the other relation (based on the join condition) are treated.

Join types
inner join
left outer join
right outer join
full outer join

Join conditions
natural
on <predicate>
using $(A_1, A_2, \ldots, A_n)$

Joined Relations – Examples

- *course* **natural right outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	*null*	*null*	*null*	CS-101

- *course* **full outer join** *prereq* **using** (*course_id*)

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*
CS-347	*null*	*null*	*null*	CS-101

Joined Relations – Examples

- *course* **inner join** *prereq* **on** *course.course_id = prereq.course_id*

course_id	*title*	*dept_name*	*credits*	*prereq_id*	*course_id*
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

- What is the difference between the above, and a natural join?
- *course* **left outer join** *prereq* **on** *course.course_id = prereq.course_id*

course_id	*title*	*dept_name*	*credits*	*prereq_id*	*course_id*
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	*null*	*null*

Joined Relations – Examples

- *course* **natural right outer join** *prereq*

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-347	*null*	*null*	*null*	CS-101

- *course* **full outer join** *prereq* **using** (*course_id*)

course_id	*title*	*dept_name*	*credits*	*prereq_id*
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101
CS-315	Robotics	Comp. Sci.	3	*null*
CS-347	*null*	*null*	*null*	CS-101

总结：连接关系

- 连接关系
  - 在From子句中的两个相邻关系之间，可以是一个逗号 —— 表示做笛卡尔积; 也可以是一个连接操作 —— 表示将它们连接成一个新的关系
  - 每个连接操作由一个连接类型和一个连接条件组成
- 连接条件
  - natural ：连接两边元组条件为同名属性相等（自然连接）
  - using (属性1, 属性2, …) ：类似自然连接，但是只限于列出的的属性相等
  - on 条件：按照指定的条件连接两边元组

连接关系

- 连接类型
  - (inner) join：内连接。结果不包含失配元组*
    - * 这里失配元组指的是因不满足连接条件，无法和其它元组相连接的元组
  - left (outer) join：左外连接。结果包含左边关系的失配元组
  - right (outer) join：右外连接。结果包含右边关系的失配元组
  - full (outer) join：全外连接。结果包含两边关系的失配元组

练习：连接关系

- 连接关系的应用
  - Stu: 学生关系
  - Class：班级关系
  - 有些学生还未分配到相应班级，有些班级也未包含任何学生

Stu

学号	姓名	班号
01	王红	C01
02	张军	C02
03	刘朝	null
04	陈平	C01

Class

班号	班名	班主任
C01	财会	老王
C02	计算机	老陈
C03	电子	老刘

练习：连接关系

每个学生的姓名和班级?

每个班级的学生人数?

列出学生和班主任之间的全部联系?

Stu right join Class on …

Stu full join Class on …

Stu left join Class on …

Stu

学号	姓名	班号
01	王红	C01
02	张军	C02
03	刘朝	null
04	陈平	C01

Class

班号	班名	班主任
C01	财会	老王
C02	计算机	老陈
C03	电子	老刘

练习：连接关系

- 每个学生的姓名和班名

?

Stu

学号	姓名	班号
01	王红	C01
02	张军	C02
03	刘朝	null
04	陈平	C01

Class

班号	班名	班主任
C01	财会	老王
C02	计算机	老陈
C03	电子	老刘

练习：连接关系

- 每个班级的学生人数?

?

Stu

学号	姓名	班号
01	王红	C01
02	张军	C02
03	刘朝	null
04	陈平	C01

Class

班号	班名	班主任
C01	财会	老王
C02	计算机	老陈
C03	电子	老刘

练习：连接关系

- 列出学生和班主任之间的全部联系？

?

Stu

学号	姓名	班号
01	王红	C01
02	张军	C02
03	刘朝	null
04	陈平	C01

Class

班号	班名	班主任
C01	财会	老王
C02	计算机	老陈
C03	电子	老刘

Views

- In some cases, it is not desirable for all users to see the entire logical model (that is, all the actual relations stored in the database.)
- Consider a person who needs to know an instructors name and department, but not the salary. This person should see a relation described, in SQL, by

  select *ID*, *name*, *dept_name*
  from *instructor*

- A **view** provides a mechanism to hide certain data from the view of certain users.
- Any relation that is not of the conceptual model but is made visible to a user as a "virtual relation" is called a **view**.

View Definition

- A view is defined using the **create view** statement which has the form

  create view *v* **as** < query expression >

  where <query expression> is any legal SQL expression. The view name is represented by *v.*
- Once a view is defined, the view name can be used to refer to the virtual relation that the view generates.
- View definition is not the same as creating a new relation by evaluating the query expression
  - Rather, a view definition causes the saving of an expression; the expression is substituted into queries using the view.

View Definition and Use

- A view of instructors without their salary

  create view ***faculty*** **as**
  select *ID*, *name*, *dept_name*
  from *instructor*

- Find all instructors in the Biology department

  select *name*
  from ***faculty***
  where *dept_name* = 'Biology'

- Create a view of department salary totals

  create view ***departments_total_salary(dept_name, total_salary)*** **as**
  select *dept_name*, **sum** (*salary*)
  from *instructor*
  group by *dept_name*;

Views Defined Using Other Views

- One view may be used in the expression defining another view
- A view relation v_1 is said to ***depend directly*** on a view relation v_2 if v_2 is used in the expression defining v_1
- A view relation v_1 is said to ***depend on*** view relation v_2 if either v_1 depends directly to v_2 or there is a path of dependencies from v_1 to v_2
- A view relation v is said to be ***recursive*** if it depends on itself.

Views Defined Using Other Views

- **create view** ***physics_fall_2017*** **as**
  select *course.course_id*, *sec_id*, *building*, *room_number*
  from *course*, *section*
  where *course.course_id* = *section.course_id*
  and *course.dept_name* = 'Physics'
  and *section.semester* = 'Fall'
  and *section.year* = '2017';

- **create view** ***physics_fall_2017***_*watson* **as**
  select *course_id*, *room_number*
  from ***physics_fall_2017***
  where *building*= 'Watson';

View Expansion

- Expand the view :

```
create view physics_fall_2017_watson as
  select course_id, room_number
  from physics_fall_2017
  where building= 'Watson'
```

- To:

```
create view physics_fall_2017_watson as
   select course_id, room_number
   from (select course.course_id, building, room_number
         from course, section
         where course.course_id = section.course_id
             and course.dept_name = 'Physics'
             and section.semester = 'Fall'
             and section.year = '2017')
    where building= 'Watson';
```

View Expansion (Cont.)

- A way to define the meaning of views defined in terms of other views.
- Let view v_1 be defined by an expression e_1 that may itself contain uses of view relations.
- View expansion of an expression repeats the following replacement step: （展开视图）

  repeat
  Find any view relation v_i in e_1
  Replace the view relation v_i by the expression defining v_i
  until no more view relations are present in e_1

- As long as the view definitions are not recursive, this loop will terminate

Materialized Views

- Certain database systems allow view relations to be physically stored.
  - Physical copy created when the view is defined.
  - Such views are called **Materialized view**:
- If relations used in the query are updated, the materialized view result becomes out of date
  - Need to **maintain** the view, by updating the view whenever the underlying relations are updated.

create materialized view

视图的优点

- 简化用户的操作
  - 例如，当经常进行某一复杂查询时，可将其固定的部分定义为一个视图，再用这个视图来重写查询，从而得到简化
- 对于同一数据，不同用户可以从不同角度观察
  - 一个关系中，不同的用户可能关注不同的部分(不同的行/列)，我们把这些部分定义成不同的视图，分配给他们
  - 表面上看，这些视图是内容不一样的“关系”。实际上，它们源自相同的基础关系，也不占用实际的存储空间。

视图的优点

- 例如，我们可以定义Class1和Class2两个视图，用户看起来它们具有不同的数据，但实际上来自同一个关系S。同时，在实际关系或者存储的数据的情况下，用户可以使用的“关系”增加了。

Class1 = ?　　**Class2 = ?**

S

学号	姓名	性别	班级
01	小张	男	1班
02	小李	女	1班
11	小陈	女	2班
12	小刘	男	2班

学号	姓名	性别	班级
01	小张	男	1班
02	小李	女	1班

学号	姓名	性别	班级
11	小陈	女	2班
12	小刘	男	2班

视图的优点

- 增强安全性
  - “知必所需”
  - 用视图来限制用户数据的访问范围。例如，在为用户定义视图时，通过选择和投影操作限制用户只能访问数据库关系中的某些行和某些列

视图的优点

- 例如，规定对于教师关系T，学生用户只能访问编号和姓名属性，而不能访问电话号码。
- 那么我们可以定义如下的视图view_forstudent，然后把它分配给学生用户，而不是把整个关系T分配给他们。

?

T

编号	教师姓名	电话
T01	老王	85213347
T02	老孙	85213109
T12	老何	85211109

编号	教师姓名
C01	老王
C02	老孙
C12	老何

视图的优点

- 思考：对教师用户“老王”，如果想限制他只能观察自己的记录，应该怎么办？

?

T

编号	教师姓名	电话
T01	老王	85213347
T02	老孙	85213109
T12	老何	85211109

编号	教师姓名	电话
T01	老王	85213347

Update of a View

- Add a new tuple to *faculty* view which we defined earlier

  insert into *faculty*

  values ('30765', 'Green', 'Music');

- This insertion must be represented by the insertion into the *instructor* relation
  - Must have a value for salary.
- Two approaches
  - Reject the insert
  - Inset the tuple

    ('30765', 'Green', 'Music', null)

    into the *instructor* relation

Some Updates Cannot be Translated Uniquely

- **create view** *instructor_info* **as**
  select *ID*, *name*, *building*
  from *instructor*, *department*
  where *instructor.dept_name*= *department.dept_name*;
- **insert into** *instructor_info*
  values ('69987', 'White', 'Taylor');
- Issues
  - Which department, if multiple departments in Taylor?
  - What if no department is in Taylor?

And Some Not at All

- **create view** *history_instructors* **as**
  select *
  from *instructor*
  where *dept_name*= 'History';
- What happens if we insert

  ('25566', 'Brown', 'Biology', 100000)

  into *history_instructors?*

View Updates in SQL

- Most SQL implementations allow updates only on simple views
  - The **from** clause has only one database relation.
  - The **select** clause contains only attribute names of the relation, and does not have any expressions, aggregates, or **distinct** specification.
  - Any attribute not listed in the **select** clause can be set to null
  - The query does not have a **group** by or **having** clause.

修改视图

- 如何修改?
  - 对视图的插入、删除、修改操作和基础关系相同。即在Insert、Delete、Update语句中，直接使用视图（代替关系）即可
- 受限制的修改
  - 视图是一个不存在的“虚拟”关系，所以对视图的修改也要转化为对基础关系的修改。
  - 这种转化不一定成功 —— 导致对视图的修改是受限的，并非所有视图都能被修改。

修改视图

- 情况一：信息缺失。如果视图定义不包括基础关系的主码，则在对视图的修改转化为对基础关系的修改时，往往会因为缺乏主码值而无法完成。例如视图class2

Create View *Class2* as
Select 姓名, 性别 From S Where 班级= '2班'

S

学号	姓名	性别	班级
01	小张	男	1班
02	小李	女	1班
11	小陈	女	2班
12	小刘	男	2班

Class2

姓名	性别
小陈	女
小刘	男

修改视图

- 情况二：信息歧义。(1) 当一个视图是来自对多个基础关系的查询时，无法决定将对视图的修改转化为对哪个基础关系的修改。例如视图View_SC

```
Create View View_SC as
Select 学号, 姓名, S.班级, 班主任 From S, C
Where S.班级=C.班级
```

S

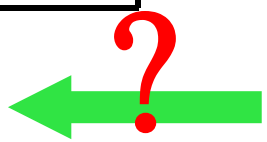
学号	姓名	性别	班级
01	小张	男	1班
02	小李	女	1班
11	小陈	女	2班

C

班级	班主任
1班	老王
2班	老何

View_SC

学号	姓名	班级	班主任
01	小张	1班	老王
02	小李	1班	老王
11	小陈	2班	老何
20	小武	3班	老章



修改视图

- 情况二：信息歧义。(2) 当视图的一个元组是“来自”关系里面的多个元组时，同样修改无法转化。例如视图Count_Class

```
Create  View   Count_Class   as
        Select      班级, count(*) as 人数
        From        S
        Group by    班级
```

S

学号	姓名	性别	班级
01	小张	男	1班
02	小李	女	1班
11	小陈	女	2班

Count_Class

班级	人数
1班	2
2班	2
3班	3

修改视图

- 出现以下情况之一，则视图是无法修改的
  - （查询的）Select 子句不包括主码
  - Select 子句中用聚集函数或算术表达式构造的新属性
  - Select 子句存在distinct关键字
  - From 子句中有多个关系
  - 有Group By子句

修改视图

- 同时满足以下条件，视图才是可修改的
  - Select 子句中包含主码
  - Select 子句中只出现原有属性，而没有用聚集函数或算术表达式构造的新属性
  - Select 子句中无distinct关键字
  - From 子句中只有一个关系
  - 无Group By子句
- 删除视图

drop view 视图名

修改视图

- 例如，下面的视图Class1是可以修改的

```
Create  View   Class1   as
Select       学号，姓名，班级
From         S
Where        班级='1班'
```

S

学号	姓名	性别	班级
01	小张	男	1班
02	小李	女	1班
11	小陈	女	2班
12	小刘	男	2班
14	小何	null	2班

Class1

学号	姓名	班级
01	小张	1班
02	小李	1班
14	小何	2班

with check option导致无法插入这条记录

总结： 视图

- 视图的本质：有名字的查询
  - 一个视图总是对应一个select查询，且有唯一的名字（视图名）
  - 查询中使用到的关系，称为基础关系。视图就是定义在基础关系上的
- 视图的表象： “虚拟”关系
  - 用户访问视图时，看到的是一个 基础关系+查询 所派生（计算）出来的“虚拟”关系，和真正的关系有相同的地方，也有不同的地方：
    - 关系（的元组）是实际存在的，即是存储在数据库中的；而视图（的元组）并不实际存在，而是通过查询“算”出来。每一次访问视图，不管在什么时候，都要通过查询“算”一次，以获得最新的内容。
    - 但在使用上，两者非常类似。在查询、添加、删除、更新操作中都可以使用视图，就像使用一个真正的关系一样。

视图

- 因此要注意以下几点

  1、当基础关系发生变化后，我们再去访问视图，看到的虚拟关系也会发生相应的变化。

  2、用户对视图的查询，系统在执行时必须转化为对基础关系的查询。

  3、用户对视图的修改，系统在执行时必须转化为对基础关系的修改。

Integrity Constraints

- Integrity constraints guard against accidental damage to the database, by ensuring that authorized changes to the database do not result in a loss of data consistency.
  - A checking account must have a balance greater than $10,000.00
  - A salary of a bank employee must be at least $4.00 an hour
  - A customer must have a (non-null) phone number

完整性的概念

- **完整性**的原义
  - 数据的完整性，指数据的**正确性、有效性和相容性**
- 大多数情况下，我们所提到的完整性，实际是指**完整性规则**
  - 为保证完整性，数据应该满足的**约束条件**
  - 又称为**完整性约束**

关系模型中的完整性

- 在关系模型中
  - 完整性又称为关系完整性，是**三要素（关系, 关系完整性, 关系操作）**之一
  - 具体地，包括三种完整性规则
    - **实体完整性；**
    - **参照完整性；**
    - **用户定义完整性；**
  - 用于解决现实世界映射到关系数据库后，产生的三类问题
    - 如何保证映射以后，一个实体是可识别（区分）的
    - 如何保证映射以后，能够从一个实体找到另一个相关联的实体，而不会出现找不到的情况
    - 如何保证映射以后，实体属性的取值是合理的

三类关系完整性规则

- **实体完整性**
  - **规则：**元组在主码的每个属性上取唯一值，且不能为空。
  - **意义：**在实体映射为元组后，不同元组靠主码来相互区分。保证每一元组/实体，可与其它元组/实体是可区分的。

班级

班号	班名
C1	1班
C2	2班
C3	3班

班级

班号	班名
C1	1班
C2	2班
C3	3班
Null	4班

三类关系完整性规则

- 要点：
  - 如果一个关系的主码由多个属性构成，那么每个属性都不能取空值

选修

学号	课程号	成绩
NULL	NULL	65
S1	C1	65
S2	C3	78
S2	C2	90
S3	C1	92

×

选修

学号	课程号	成绩
NULL	C1	65
S1	C1	65
S2	C3	78
S2	C2	90
S3	C1	92

×

Constraints on a Single Relation

- **not null**
- **primary key**
- **unique**
- **check** (P), where P is a predicate

Not Null Constraints

- **not null**
  - Declare *name* and *budget* to be **not null**

    name **varchar**(20) **not null**
    budget **numeric**(12,2) **not null**

Unique Constraints

- **unique** (A_1, A_2, …, A_m)
  - The unique specification states that the attributes A_1, A_2, …, A_m form a candidate key.
  - Candidate keys are permitted to be null (in contrast to primary keys).

Referential Integrity

- Ensures that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation.
  - Example: If "Biology" is a department name appearing in one of the tuples in the *instructor* relation, then there exists a tuple in the *department* relation for "Biology".
- Let A be a set of attributes. Let R and S be two relations that contain attributes A and where A is the primary key of S. A is said to be a **foreign key** of R if for any values of A appearing in R these values also appear in S.

Example

班级

班号	班名
C1	1班
C2	2班
C3	3班

学生

班号	学号	姓名	年龄
C1	S1	小王	22
C2	S2	小张	20
C2	S3	小李	24
Null	S4	小孙	23
C4	S5	小何	22

Referential Integrity (Cont.)

- Foreign *keys can be* specified as part of the SQL **create table** statement

  foreign key (*dept_name*) **references** *department*

- By default, a foreign key references the primary-key attributes of the referenced table.
- SQL allows a list of attributes of the referenced relation to be specified explicitly.

  foreign key (*dept_name*) **references** *department* (*dept_name*)

Cascading Actions in Referential Integrity

- When a referential-integrity constraint is violated, the normal procedure is to reject the action that caused the violation.
- An alternative, in case of delete or update is to cascade

```
create table course (
        (…
         dept_name varchar(20),
         foreign key (dept_name) references department
              on delete cascade
              on update cascade,
          . . .)
```

- Instead of cascade we can use :
  - **set null**,
  - **set default**

外部码约束

- 参照动作
  - 说明当某个主码值被删除/更新时（这个主码值在被参照关系中），如何处理对应的外部码值（这些外部码值在参照关系中）
  - **RESTRICT 方式**：仅当没有任何对应的外码值时，才可以删除/更新这个主码值，否则系统拒绝执行此操作
  - **CASCADE 方式**：连带将所有对应的外码值一块删除/更新（删除外码值，实际上就是将所在的元组删除掉）
  - **SET NULL 方式**：将所有对应的外码值设为空值

三类关系完整性规则

- 参照完整性
  - **规则：**如果关系R的外部码对应关系S的主码，则R每个元组在外部码上的取值必须满足：
    - **要么等于空值**
    - **要么等于某个对应的主码值（S某个元组的主码值）**
  - **意义：**元组/实体的外部码，说明跟另外一个元组/实体（的主码）相联系的。保证这一联系有意义，不会联系不存在的元组/实体。

Integrity Constraint Violation During Transactions

- Consider:

  create table *person* (
  ID **char**(10),
  name **char**(40),
  mother **char**(10),
  father **char**(10),
  primary key *ID,*
  foreign key *father* **references** *person,*
  foreign key *mother* **references** *person*)

- How to insert a tuple without causing constraint violation?
  - Insert father and mother of a person before inserting person
  - OR, set father and mother to null initially, update after inserting all persons (not possible if father and mother attributes declared to be **not null**)
  - OR defer constraint checking

课堂练习

- **依次**执行如下操作， 哪些能够成功?
  1. 零件关系: **添加** (3, 绿, null)
  2. 供应商关系: **添加** (null, 四化, 广州)
  3. 供应商关系: **添加** (E, 北电, 广州)
  4. 零件关系: **修改** (2, 白, A) 为 (2, 黑, F)
  5. 供应商关系: **删除** (A, 红星, 北京)
  6. 零件关系: **修改** (3, 蓝, B) 为 (3, 蓝, E)

零件

零件号	颜色	供应商号
1	红	B
2	白	A
3	蓝	B

供应商

供应商号	供应商名	所在城市
A	红星	北京
B	宇宙	上海
C	黎明	广州
D	立新	深圳

三类关系完整性规则

- 用户定义完整性
  - **规则：**用户根据具体的应用环境定义
    - 例如
      - 年龄的取值范围为0到100，性别只能是“男”或“女”
      - 职工编号是4位整数
  - **意义：**
    - 保证元组/实体的属性取值合理，反映现实世界的真实情况
    - 反映了程序编制的要求

The check clause

- The **check** (P) clause specifies a predicate P that must be satisfied by every tuple in a relation.
- Example: ensure that semester is one of fall, winter, spring or summer

```
create table section
    (course_id varchar (8),
     sec_id varchar (8),
     semester varchar (6),
     year numeric (4,0),
     building varchar (15),
     room_number varchar (7),
     time slot id varchar (4),
     primary key (course_id, sec_id, semester, year),
     check (semester in ('Fall', 'Winter', 'Spring', 'Summer')))
```

练习

- 学生的Ssex只能是“男” 或 “女”
- 学生的年纪Sage 在 0到100之间
- Create TABLE student (
  - Sno CHAR(10) primiary key,
  - Ssex CHAR (2) ？
  - ……
  - Sage SMALLINT ？
  -)

练习

- 例：CET4是一个记录四级成绩的表，有两个属性：“result”和“grade”。如何保证“grade”的值（及格、不及格两档）和“result”的值能够正确对应?
  - 420 到 710为及格，0-419为不及格

```
create table CET4
    (sno     VARCHAR (20),
     grade   VARCHAR (10),
     result  FLOAT,
     primary key (sno)
     )
```

语句： Alter Table CET4
        Add Constraint C1 CHECK (?)

Complex Check Conditions

- The predicate in the check clause can be an arbitrary predicate that can include a subquery.

  check (*time_slot_id* **in** (**select** *time_slot_id* **from** *time_slot*))

  The check condition states that the time_slot_id in each tuple in the *section* relation is actually the identifier of a time slot in the *time_slot* relation.

  - The condition has to be checked not only when a tuple is inserted or modified in *section* , but also when the relation *time_slot* changes

Assertions

- An **assertion** is a predicate expressing a condition that we wish the database always to satisfy.
- The following constraints, can be expressed using assertions:
- For each tuple in the *student* relation, the value of the attribute *tot_cred* must equal the sum of credits of courses that the student has completed successfully.
- An instructor cannot teach in two different classrooms in a semester in the same time slot
- An assertion in SQL takes the form:

  create assertion <assertion-name> **check** (<predicate>);

断言

- 断言就是**一个谓词**（条件），施加在较大范围，比如可以是整个数据库的所有数据
  - 主码约束、检查约束等作用范围较小（一个元组或属性内部），可以看作是**“小断言”**
  - 如果作用范围比较大，比如约束几个关系之间必须满足什么条件，就要**“大断言”**

断言

- 定义

create assertion 断言名 *check* (条件)

  - 断言创建以后，系统要对每个可能违反该断言的修改操作进行检查。这种检查会带来**巨大的系统负载**，因此应该谨慎使用断言。
  - 所以实际上很少数据库软件支持断言，而倾向用其它的等价方法，比如**触发器**。

断言

- 例：2班每个学生选修的课程数不得超过10门

```
create assertion sc_assert1 check
 (10 >= all
     (select count(课程号)
     from S, SC
     where S.学号 = SC.学号 and S.班级= '2班'
     group by S.学号)
 )
```

子查询：2班学生选修的课程数

S

学号	姓名	班级
…	…	…

SC

学号	课程号	成绩
…	…	…

C

课程号	名称	学时
…	…	…

断言

- 例：1班的学生全部选修数据库课程⇔不存在未选修数据库课程的1班学生

```
create assertion sc_assert2 check
    (
      not exists (? )
    )
```

子查询：
未选修数据库的1班学生

子子查询：
选修数据库的学号

S

学号	姓名	年级
…	…	…

SC

学号	课程号	成绩
…	…	…

C

课程号	名称	学时
…	…	…

Built-in Data Types in SQL

- **date:** Dates, containing a (4 digit) year, month and date
  - Example: **date** '2005-7-27'
- **time:** Time of day, in hours, minutes and seconds.
  - Example: **time** '09:00:30' **time** '09:00:30.75'
- **timestamp:** date plus time of day
  - Example: **timestamp** '2005-7-27 09:00:30.75'
- **interval:** period of time
  - Example: interval '1' day
  - Subtracting a date/time/timestamp value from another gives an interval value
  - Interval values can be added to date/time/timestamp values

Large-Object Types

- Large objects (photos, videos, CAD files, etc.) are stored as a *large object*:
  - **blob**: binary large object -- object is a large collection of uninterpreted binary data (whose interpretation is left to an application outside of the database system)
  - **clob**: character large object -- object is a large collection of character data
- When a query returns a large object, a pointer is returned rather than the large object itself.

User-Defined Types

- **create type** construct in SQL creates user-defined type

  create type *Dollars* **as numeric (12,2) final**

- Example:

  create table *department*
  (*dept_name* **varchar** (20),
  building **varchar** (15),
  budget Dollars);

Domains

- **create domain** construct in SQL-92 creates user-defined domain types

  create domain *person_name* **char**(20) **not null**

- Types and domains are similar. Domains can have constraints, such as **not null**, specified on them.
- Example:

  create domain *degree_level* **varchar**(10)
  constraint *degree_level_test*
  check (**value in** ('Bachelors', 'Masters', 'Doctorate'));

Index Creation

- Many queries reference only a small proportion of the records in a table.
- It is inefficient for the system to read every record to find a record with particular value
- An **index** on an attribute of a relation is a data structure that allows the database system to find those tuples in the relation that have a specified value for that attribute efficiently, without scanning through all the tuples of the relation.
- We create an index with the **create index** command

  create index <name> **on** <relation-name> (attribute);

Index Creation Example

- **create table** *student*
  (*ID* **varchar** (5),
  name **varchar** (20) **not null**,
  dept_name **varchar** (20),
  tot_cred **numeric** (3,0) **default** 0,
  primary key (*ID*))
- **create index** *studentID_index* **on** *student*(*ID*)
- The query:

  select *
  from *student*
  where *ID* = '12345'

  can be executed by using the index to find the required record, without looking at all records of *student*

总结：索引

- 索引是关系型数据库的一个基本概念。
- 数据库的索引类似图书的索引，能够使数据库程序不用浏览整个表，就可以找到表中的数据。
- 索引是一个表中所包含的值的列表，它说明了表中包含各个值的行所在的存储位置。
- 用户可以利用索引快速访问数据库表中的特定信息。
- 但使用索引存储地址将占用磁盘空间，同时在数据维护时，也将花费一定的时间。因此要合理设计索引。

Authorization

- We may assign a user several forms of authorizations on parts of the database.
  - **Read** - allows reading, but not modification of data.
  - **Insert** - allows insertion of new data, but not modification of existing data.
  - **Update** - allows modification, but not deletion of data.
  - **Delete** - allows deletion of data.
- Each of these types of authorizations is called a **privilege**. We may authorize the user all, none, or a combination of these types of privileges on specified parts of a database, such as a relation or a view.

Authorization (Cont.)

- Forms of authorization to modify the database schema
  - **Index** - allows creation and deletion of indices.
  - **Resources** - allows creation of new relations.
  - **Alteration** - allows addition or deletion of attributes in a relation.
  - **Drop** - allows deletion of relations.

Authorization Specification in SQL

- The **grant** statement is used to confer authorization

  grant <privilege list> **on** <relation or view > **to** <user list>
- <user list> is:
  - a user-id
  - **public**, which allows all valid users the privilege granted
  - A role (more on this later)
- Example:
  - **grant select on** *department* **to** Amit, Satoshi
- Granting a privilege on a view does not imply granting any privileges on the underlying relations.
- The grantor of the privilege must already hold the privilege on the specified item (or be the database administrator).

Privileges in SQL

- **select**: allows read access to relation, or the ability to query using the view
  - Example: grant users U_1, U_2, and U_3 **select** authorization on the *instructor* relation:

    grant select on *instructor* **to** U_1, U_2, U_3
- **insert**: the ability to insert tuples
- **update**: the ability to update using the SQL update statement
- **delete**: the ability to delete tuples.
- **all privileges**: used as a short form for all the allowable privileges

Revoking Authorization in SQL

- The **revoke** statement is used to revoke authorization.

  revoke <privilege list> **on** <relation or view> **from** <user list>

- Example:

  revoke select on *student* **from** U_1, U_2, U_3

- <privilege-list> may be **all** to revoke all privileges the revokee may hold.
- If <revokee-list> includes **public,** all users lose the privilege except those granted it explicitly.
- If the same privilege was granted twice to the same user by different grantees, the user may retain the privilege after the revocation.
- All privileges that depend on the privilege being revoked are also revoked.

Roles

- A **role** is a way to distinguish among various users as far as what these users can access/update in the database.
- To create a role we use:

  create a role <name>
- Example:
  - **create role** instructor
- Once a role is created we can assign "users" to the role using:
  - **grant** <role> **to** <users>

Roles Example

- **create role** instructor;
- **grant** *instructor* **to** Amit**;**
- Privileges can be granted to roles:
  - **grant select on** *takes* **to** *instructor*;
- Roles can be granted to users, as well as to other roles
  - **create role** *teaching_assistant*
  - **grant** *teaching_assistant* **to** *instructor*;
    - *Instructor* inherits all privileges of *teaching_assistant*
- Chain of roles
  - **create role** *dean*;
  - **grant** *instructor* **to** *dean*;
  - **grant** *dean* **to** Satoshi;

Authorization on Views

- **create view** *geo_instructor* **as**
  (**select** *
  from *instructor*
  where *dept_name* = 'Geology');
- **grant select on** *geo_instructor* **to** *geo_staff*
- Suppose that a *geo_staff* member issues
  - **select** *
    from *geo_instructor*;
- What if
  - *geo_staff* does not have permissions on *instructor?*
  - Creator of view did not have some permissions on *instructor?*

Other Authorization Features

- **references** privilege to create foreign key
  - **grant reference** (*dept_name*) **on** *department* **to** Mariano;
  - Why is this required?
- transfer of privileges
  - **grant select on** *department* **to** Amit **with grant option**;
  - **revoke select on** *department* **from** Amit, Satoshi **cascade**;
  - **revoke select on** *department* **from** Amit, Satoshi **restrict**;
  - And more!

End of Chapter 4